

totalidad 8/20/18

① V OF

???

(a) Se puede obtener una matriz de transición, haciendo un ciclo a través de una matriz proveniente del algoritmo de Floyd

(b) Se puede implementar una pila estática que en vez de arrancar en 0 o en n (v) se va llenando el vector desde el "nivel" hasta el principio 0

② Dado un árbol binario proveniente de un bosque, hallar el nivel mayor

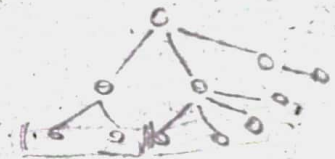
```
int nivelMayorBosque(arbol a){
    int max = 0, nivel;
    while(a != null){
        nivel = nivelMayor(a->izq, 1);
        if(nivel > max)
            max = nivel;
        a = a->der;
    }
    return max;
}
```

```
int nivelMayor(arbol a, int nact){
    int maxIzq, maxDer;
    if(a){
        nact++;
        maxIzq = (a->izq, nact);
        maxDer = (a->der, nact);
        if(maxIzq > maxDer)
            return maxIzq;
        else
            return maxDer;
    }
    else
        return nact;
}
```

③ Generar una lista doble en la cual cada nodo posea la suma de los elementos ubicados en niveles pares con cantidad de elementos pares de un árbol n-ario, no usar variables auxiliares. → no se pudo usar de cosa ???

```
void generarLD (lista* ld, arbol a, posición p, int nact){
    posición, e;
    if(!nodo(p)){
        if(nact % 2 == 0){

```



seguir la misma idea
que en el del 24/8/20
asumiendo que el nivel en el nodo

```
nodo{
    int lones, nivel, cant;
    nodo *ant, *sig; nodo;
}
```

④ Generar una cola circular que en cada nodo contenga dato, y el grado de entrada de un grafo dirigido de N vértices que se encuentra en una lista de adyacencia no utilizar estructuras auxiliares

```
void serial (TCola *c, Tlista A[], int N){
    int i, j, gRE;
    TElemento *aux;
    Tlista auxl; inicia(c);
    for (i=0, i<N, i++){
        gRE=0;
        for (j=0, j<N, j++){
            if (j!=i){
                aux=A[j];
                while (aux!=NULL && aux->dato!=i+j){
                    aux=aux->sig;
                }
                if (aux!=NULL && aux->dato==i){
                    gRE++;
                }
            }
            K.vertex=i+j;
            K.gRE=gRE;
            pone(c, K);
        }
    }
}
```

```
void pone (TCola *c, TElemento K){
    if (((*c).vet != MAX-1 && (*c).Pri != 0) || (*c).vet+1 == (*c).Pri)) {
        if ((*c).Pri == -1){
            (*c).Pri = 0;
            (*c).vet = 0;
        }
        else {
            if ((*c).vet == MAX-1){
                (*c).vet = 0;
            }
            else {
                (*c).vet++;
            }
        }
        (*c).datos[(*c).vet] = K;
    }
}
```

```
void inicia (TCola *c){
    (*c).Pri = -1;
    (*c).vet = -1;
}
```


Totalizador 2012/2021

④ VOF

- ① En caminata final tras ejecutar Floyd (w) todos los elementos de la diagonal principal son 0 a menos que haya bucle (v) ^{si se el camino mas corto de un nodo a mismo nodo existe, hay bucle}
- ② una cola implementada dinámicamente de nodo circular, permite apreciar a pe los nodos que no tienen datos significativos no existe la implementación dinámica

② LD

Zona (1, 10) → si red se refiere a zona

col causa/radar (si cuenta con c es conura y si con R radar, cadu)

pila de infracciones (podria estar vacia)

Fecha Hora (cadena de 10 MMDD-HH:MM)

Patente (cadena de 7)

un nodo x cada cam y Rad

- ① al tener la cantidad de radares se en zonas pares van hacia infracciones el 1º y 2º las pilas no pueden estar se * Hacer con una función que trate recursivamente la pila

typedef struct nodo {

int zona;

char cad[5];

Tpila infracciones

struct nodo * ant, * sig;

typedef struct

char H[11], patente[8];

Telemento x;

typedef struct {

Telemento datos[MAX];

int tope / Tpila;

typedef struct

nodo * pri, * ult; } + w real

int cantInfracciones (TustaD LD, char Fecha[10])

int cant = 0;

Tnodo act;

Telemento x;

Tpila aux;

act = LD.pri;

while (act != NULL) {

if (act->zona[1] * 2 == 0 && act->zona[0] == '2') {

pushP(&aux);

while (!vacioP(act->infracciones)) {

popP(&act->infracciones, &x); popP(&aux, x);

if (strcmpP(x, H, Fecha) != 0)

cont++;

act = act->sig;

while (!vacioP(aux)) {

popP(&aux, x);

act = act->sig;

return cant;

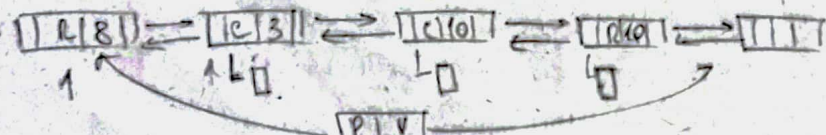
pila zona

reserva sensores

mem

infracciones en

ambos



⑥ Generar una lista circular ordenada x la cont de infracciones en el zona solo para aquellas que contienen infracciones por radar y por camaras (la pica red verde). Cada nodo tendra

zona (no se repite)
cont infracciones R
cont infracciones C

Para cada zona del sistema tiene infracciones.
tarea en R como en C y luego ordenar la LC
x el total → recorrer 10 veces ???

void (*tutor) (LD, Tustac LC)

int cantR, cantC, i;

Tustac act, ant, nuevo;

nodos *aux;

TElemento x;

Reg v[10];

aux = LD.Pri;

while (aux != NULL)

v[aux->zona-1].R = 0;

v[aux->zona-1].C = 0;

while (!Vacia (aux->infracciones))
sacaPlg (aux->infracciones, i);

if (aux->cod = 'R')

v[aux->zona-1].R += 1;

else

v[aux->zona-1].C += 1;

aux = aux->sig;

for (i = 0, i < 10, i++)

if (v[i].R != 0 || v[i].C != 0)

nuevo = (Tustac) malloc (sizeof (Tustac));

nuevo->zona = i+1;

nuevo->cantR = v[i].R;

nuevo->cantC = v[i].C;

v[i].R = 0;

*LC = nuevo;

nuevo->sig = NULL;

else

if ((*LC)->cantR + (*LC)->cantC < v[i].R + v[i].C)

nuevo->sig = (*LC)->sig;

(*LC)->sig = nuevo; (*LC) = nuevo;

else

ante = *LC, act = (*LC)->sig;

while (act != *LC && act->cantR + act->cantC < v[i].R + v[i].C)

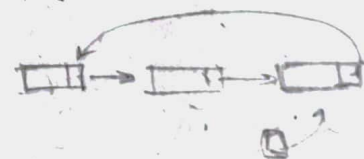
ant = act; act = act->sig;

act->sig = nuevo;

nuevo->sig = act;

Salvo que haga el nodo sig
y al final de recorrerla
LD, recorrala LC y elimine
los que no tengan infracciones
en ambos, pero tengo que
ordenarla x la cantidad de
infracciones

USAR UN VECTOR
Y RECORRER una
solo vez la LD
y a partir de
vector hacer la
LC




```

② void sacarP(TPila *P, TElementoP *x){
    if ((*P).tope != -1){
        *x = (*P).datos[(*P).tope];
        ((*P).tope) --;
    }
}

```

```

void cargarP(TPila *P, TElementoP *x){
    if ((*P).tope != MAX-1){
        (*P).datos[+ ((*P).tope)] = *x;
    }
}

```

③ Se tiene un árbol N -ario de cadenas se desea una función en tercer nivel que determine cuántas cadenas de longitud impar que comiencen y terminen en vocal se encuentran en núcleos impares

```

int contarImpares (Arbol A, Posicion P, int nact){
    Posicion Ci; char aux[10];
    int cant;
    if (!Nulo(P)){
        cant = 0; strcpy(aux, INFO[P, n]);
        if ((nact % 2 != 0) && strlen(aux) % 2 != 0 && (esVocal(aux[0]) && esVocal(aux[strlen(aux)-1])){
            cant ++;
        }
        C = HijoPos(P, A);
        while (!Nulo(C)){
            cant += contarImpares(A, C, nact+1);
            C = HermanoDer(C, A);
        }
        return cant;
    }
    else
        return 0;
}

```

④ Se tiene un grafo $G = (V, E)$ con $V = \{W\}$ y aristas ponderadas, almacenadas en una matriz de adyacencia y un ABR de enteros. Determinar de forma recursiva sobre la matriz si para cada vertice de G que tenga bucle se verifica que su grado de entrada se encuentra en el ABR. Definir tipos.

```

int verifica (int A[][MAX], int n, int i, int j, int g, int s){
    if (g < n){
        if (i == 0 && A[g][g] != 0){
            return verifica(A, n, n, i, g+1, 0);
        }
        else if (i < n){
            return verifica(A, n, i, g, g+A[i][g]);
        }
        else if (A[i][g] > 0){
            return verifica(A, n, i, g+1, 0);
        }
        else
            return 1;
    }
    else
        return 0;
}

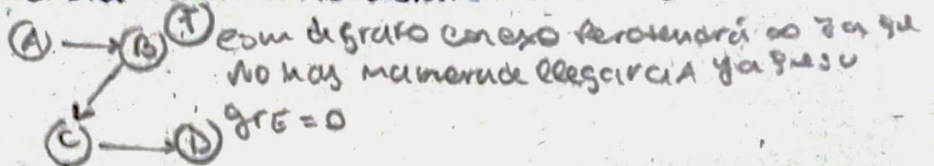
int EstaEnABR(int a){
    if (a != NULL){
        if (a->dato == 2){
            return 1;
        }
        else if (a->dato < x){
            return EstaEnABR(a->der);
        }
        else
            return (a->der);
    }
    else
        return 0;
}

```


Tutoría de 2/8/2021

① Indicar V o F

② La matriz final de Floyd no tendrá infinitos si el grafo es conexo y tiene ^{al} menos $N-1$ aristas siendo $|V|=N$



④ Después de una inserción en un AVL, si se detecta un nodo desbalanceado se debe aplicar una rotación por se elige de acuerdo a la posición o nivel en el que se ha insertado el nodo. ^⑤ se elige de acuerdo al camino que se sigue por la rama más larga que se desbalancea.

⑤ En un complejo con varios salones de fiesta se han tomado reservas que se almacenan en una lista simplemente enlazada donde cada nodo contiene:

- codsalon (cod 3 ordenado)
- capacidad salon
- sublista de reservas
 - AgN del nombre de la reserva
 - Cantidad invitadas
 - Mail de contactos
 - reserva de la reserva $\rightarrow$ ordenado descendientemente, no se repite

⑥ Se ha recibido una reserva para un día y con d personas, se solicita mostrar los salones que hay disponibles para dicha reserva considerando además de la disponibilidad que la capacidad del salón no se sea ocupada hasta el 50%.

⑦ el día D y mes M se deberá hacer una desinfección, dar de baja a todas las reservas de ese día almacenadas en un archivo BADA0.TXT el AgN y mail de las personas que tienen reservas ese día.

⑧ while ($L \neq \text{NULL}$) {

```
if (L->capacidad * 0.5 > Invitados) {
    act = L->reservas; (strcpy(MD, MD); strcat(MD, D));
    while (act != NULL && strcmp(act->MD, MD) > 0) {
        act = act->sig;
    }
    if ((act != NULL && strcmp(act->MD, MD) < 0) || act == NULL) {
        printf("El salon %s esta disponible para la fecha solicitada", L->codsalon);
    }
    L = L->sig;
}
```

⑨ if ($(\text{arch} = \text{fopen}(\text{"BADA0.TXT"}, \text{"wt"})) \neq \text{NULL}$) {

```
while (L != NULL) {
    act = L->reservas; act = NULL;
    while (act != NULL && strcmp(act->MD, MD) > 0) {
        act = act->sig;
    }
    if (act != NULL && strcmp(act->MD, MD) == 0) {
        fprintf(arch, "%s, %s\n", act->AgN, act->mail);
        if (act == L->reservas) {
            L->reservas = act->sig;
        }
        else {
            act->sig = act->sig;
        }
    }
}
```

```
free(act);
L = L->sig;
fclose(arch);
}
else {
    printf("Error al abrir el archivo");
}
```

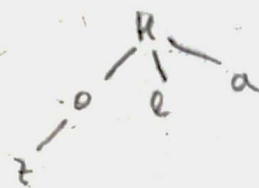

3) Se tiene un árbol n-ario que contiene caracteres determinar mediante una función en recursión, si existe algún nodo que considerando su clave y la de sus hijos forme (en orden y secuencia) la palabra X (dato de entrada):

```

int Formax (arbol a, posición p, char x[]) {
    posición c; char cad[7]; int forma;
    if (!Nulo(p)) {
        strcpy(cad, info(p, a));
        forma = 0;
        c = hijoMaximo(p, a);
        while (!Nulo(c) && !forma) {
            strcat(cad, info(c, a));
            forma = Formax(a, c, x);
            c = hermanoDer(c, a);
        }
        return (forma || strcmp(cad, x) == 0);
    }
    else
        return 0;
}

```

X = "Rolo"



4) Dado el siguiente código incompleto y con errores se pretende narrar los miembros desde el ~~inicio~~ inicial, el for es correcto y el vector de la lista de adyacencia tiene índice con significado. Se pide completar y corregir el código para resolverlo como se pretende

```

#define N 10
typedef struct node NODO {
    int dato;
    struct nodo *sig;
} NODO;

```

```

typedef NODO *TLISTA;

```

```

void Proximidad (TLISTA grafo[], int inicial, int n) {

```

```

    int w[N] = {0};

```

```

    TPila pila;

```

```

    Elemento ep;

```

```

    TLISTA aux;

```

```

    iniciaP(pila);

```

```

    pilaP(&pila, inicial); w[inicial] = 1;

```

```

    printf("%d", inicial);

```

```

    while (!todovistos(w, n) && !w[inicial]) {

```

```

        sacar(&pila, &ep); // consigna(pila)

```

```

        aux = grafo[ep];

```

```

        while (w[aux->dato] aux != NULL && w[aux->dato])

```

```

            aux = aux->sig;

```

```

        if (aux != NULL) {

```

```

            pilaP(&pila, &aux->dato);

```

```

            w[aux->dato] = 1;

```

```

            printf("%d", aux->dato);

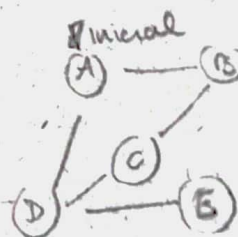
```

```

        }
    }
}

```

A	- D - B
B	- A - C
C	- D - B
D	- A - E - C
E	- D



A - D - E - C - B

① VOF

(a) una cola circular, podría implementarse e efectuando corrimientos, lo que implicaría modificar el algoritmo de inserción como el de eliminación. ^{② solo hace falta modificar los índices de PI y PT pero si hubiera que modificar los datos}

(b) si luego de ejecutar el algoritmo de Floyd, la matriz final A contiene infinitos significa que el grafo es no conexo. $\rightarrow$ si es un grafo no necesariamente tiene pesos en el grafo si

② Superar una LDE en la que cada nodo contiene un número entero N (positivos sin repetir) y una sublista con valores enteros P se superan sin divisores de N

① Dado k verificar si todos los elementos de la sublista del nodo que contiene k son sus divisores. Eliminar los que no sean, informar la situación

② Dado x generar un archivo de texto DIVI.TXT con los valores de la lista que sean Pares y tener exactamente x divisores. Colocar un valor por línea

① void verificar (TlistAD LD, int k)

nodod *act;

sublista acts; acts;

act = (LD).Pri;

while (act != NULL && act->dato != k)

act = act->sig;

if (act != NULL && act->dato == k)

acts = act->divisores; acts = NULL;

while (acts != NULL)

if (k % acts->dato != 0)

if (acts == act->divisores)

act->divisores = acts->sig;

free(acts);

acts = act->divisores;

else

acts->sig = act->sig;

free(acts);

acts = acts->sig;

else

acts = acts->sig;

}

}

* while (acts != NULL)

if (acts->dato % k == 0)

fprintf (arq, "%d", acts->dato);

acts = acts->sig;

fclose(arq);

else

printf ("Error al abrir el archivo");

② void generaArq (TlistAD, int x)

TlistAD act;

sublista acts;

int cant;

File *arq;

if ((arq = fopen("DIVI.TXT", "wt")) != NULL)

act = LD.Pri;

while (act != NULL && act->dato != x)

act = act->sig;

acts = act->divisores;

③ se tiene un árbol N -ÁRD AN de números enteros, determinar en qué nivel se encuentra la primera aparición de K en AN según el recorrido inorden del árbol (K dato de entrada)

④ Dado un digrafo con aristas ponderadas implémentando en una lista de adyacencia, y una pila P en la que cada elemento contiene un peso y un vértice se pide definir T(R) y desarrollar una solución recursiva sobre la pila que elimine de M a aquellos vértices cuyo peso sea negativo que el costo total de las aristas que vinculan con él que no son bucles.

```

int CostTotal (Tree A[], int N, int U) {
    int i, Cost0;
    Cost0 = 0;
    act = A[U-1];
    while (act != NULL) {
        IF (act->vertex != U)
            Cost0 += act->Cost0;
        act = act->sig;
    }
    for (i = 0; i < N; i++)
        IF (i != U-1) {

```

Escaneado con CamScanner

"Modelo Provisorio"

- ① Se tiene una matriz de $N \times N$ naturales que representa un AMM construido sobre un grafo. Desarrollar una selección recursiva sobre la matriz que elimine en la cola los vértices y su grado en el AMM, sólo para aquellos vértices que grado > 1 en el AMM, retornar además el costo del AMM

```
void creaCola(int A[][MAX], int N, int l, int f, int g, TNode *c)
{
    TNode *p;
    if (l < N) {
```

```
        if (g < N) {
```

```
            creaCola(A, N, l, g+1, g+A[l][g], c);
```

```
        else {
```

```
            if (g > 1) {
```

```
                x.vertice = i; x.grado = g;
```

```
                poneC(c, x);
```

```
                creaCola(A, N, l+1, 0, 0, c);
```

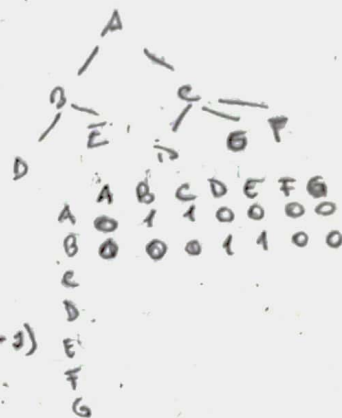
```
            }
            poneC
```

```
            if ((c).vet != MAX-1) {
```

```
                if ((c).Pri == -1) {
```

```
                    (*c).Pri = 0;
```

```
                    (c).datos[+(c).vet] = x;
```



- ② una lista simplemente enlazada se generó luego de haber analizado un texto en cada nodo:

{ letra (ordenada, pueden no estar todas las letras del alfabeto)

{ subestructura de palabras

{ palabra → ordenada

{ apariciones (>0)

- ③ Dada una palabra que se incluye en lista enumerada

- ④ Generar una lista circular que contenga en cada nodo, sólo las palabras que comienzan con vocal y aparecieron más de cinco veces

{ palabras (ordenada, y comienza con vocal)

{ apariciones (>5)

```
void eliminaPal (TNode *L, char pal[]) {
```

```
    struct act, ant;
```

```
    while (L != NULL && L->letra != pal[0]) {
```

```
        L = L->sig;
```

```
    if (L != NULL && L->letra == pal[0]) { // la encontramos la letra
```

```
        act = L->palabras; ant = NULL;
```

```
        while (act != NULL && strcmp(act->palabra, pal) < 0) {
```

```
            ant = act;
```

```
            act = act->sig;
```

```
        if (act != NULL && strcmp(act->palabra, pal) == 0) {
```

```
            if (act->palabras)
```

```
                l->palabras = act->sig;
```

```
            else act->sig = act->sig; free(act);
```



```
void General (Tusor *Lc, Tusor L){
```

```
    susista act;
```

```
    Tusor nuevo, act, acti;
```

```
    while (L != NULL){
```

```
        IF (Es vocal(L->letra))
```

```
        { act = L->palabras;
```

```
        while (act != NULL){
```

```
            IF (act->aparciones > 0){
```

```
                nuevo = (Tusor) malloc (sizeof (Tusor));
```

```
                strcpy (nuevo->palabra, act->palabra);
```

```
                nuevo->aparciones = act->aparciones;
```

```
                IF (*Lc == NULL){
```

```
                    (*Lc) = nuevo;
```

```
                    nuevo->sig = nuevo;
```

```
                else
```

```
                    IF (strcmp ((*Lc)->palabra, nuevo->palabra) < 0)
```

```
                        nuevo->sig = (*Lc)->sig;
```

```
                        (*Lc)->sig = nuevo;
```

```
                        (*Lc) = nuevo;
```

```
                    else{
```

```
                        act = (*Lc)->sig;
```

```
                        while (act != (*Lc) && strcmp (act->palabra, nuevo->palabra) < 0){
```

```
                            act = act;
```

```
                            act = act->sig;
```

```
                        act->sig = nuevo;
```

```
                        nuevo->sig = act;
```

```
                    }
```

```
                }
            }
        }
    }
    L = L->sig;
```

③ Dado un árbol n-ario de enteros, determinar mediante una función int cántas hojas en el nivel k contienen un 0.

```
int ContCeros (arbol a, posición p, int k, int nact){
```

```
    posición e, int cont;
```

```
    IF (!vuelo(p))
```

```
    { IF (nivel(k))
```

```
        return ContCeros (a, hijoNro (p, a), k, nact+1) + ContCeros (a, hermanoDer (p, a), k, nact);
```

```
    } else{
```

```
        return ContCeros (a, hermanoDer (p, a), k, nact) + (nivelNroNro (p, a) && vuelo(a) == 0);
```

```
    } else
```

```
        return 0;
```


totalizador 19/12/2022

① VOF

- ① Actualizar la ejecución del algoritmo de Floyd sobre un digrafo, si el mismo es no conexo habrá posiciones con valor 0 y otras con valor infinito (F)
- ② Al insertar una clave en un árbol AVL, se deben calcular los f.e. desde la raíz hasta la clave insertada en el proceso inserción (a la ida) (V)
- ③ no necesariamente tendrá a ceros en la diagonal eso no sirve para decir si el grafo fuera acíclico pero en ese caso es no conexo es decir que en ciertos componentes aparecen 0
- ④ se debe volver a calcular los Fe de todos para saber si todo el árbol se desbalancea

② LC de normas (0..20) y empresas (0..39) → se puede certificar en más de 1 ISO

① Dada una norma eliminar todas las empresas que hayan certificado N en el mes MM, si se pide sin eliminar la norma. Generar un DAT con la empresa eliminada nombre, y fecha de certificación (ISO)

LC { Norma (ordenada, no se repite)
Nombre Norma (cadena 8)
Sublista de empresas
 { Nro Empresa
 Fecha de certificación (cadena MM)
 ordenada x fecha }

② Generar una matriz de 21x40 caracteres norma i, empresa j entonces figura S/N si la empresa j se certifica en la norma i. Hallar de forma recursiva mediante una función int la cantidad de empresas que han certificado todas las normas

③ void Genera (char M[][MAX], Tlistac LC)

int i, j; Tlistac act; Sublista acts;

if (!LC)

act = LC;

do

act = act->sig;

acts = act->Empresas;

while (acts != NULL)

M[act->Norma][acts->NE] = 'S';

acts = acts->sig;

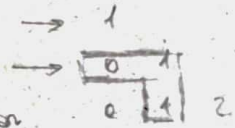
while (act != LC->sig)

act = act->sig;

while (act != LC->sig)

};

// Inicializo la matriz con 'N'



(...)

a. 1 int CertificaTodas (char M[][MAX], int i, int j)

if (i < MAX)

if (j < MAX)

if (M[i][j] == 'S')

return CertificaTodas(M, i, j+1)

else return CertificaTodas(M, i+1, 0)

else return CertificaTodas(M, i+1, 0) + 1

else

return 0;

⑥ void elimina (Tlistac *LC, char mm[3], int N)

Reg X;

Tlistac act, ant;

Subestructa acts, ants;

if ((!LC) && (arch=fopen("ELIM.DAT","wb")) != NULL)

act = *LC;

act = act;

act = act->sig;

while (act != *LC && act->norma < N)

if (act->norma == N)

acts = act->empresas;

ants = NULL;

while (acts != NULL)

strcpy (aux, acts->fecha);

strlen (aux);

if (strcmp (aux, mm) == 0) // coincide con el mes, lo elimino

if (acts == act->empresas)

act->empresas = acts->sig;

free (acts);

acts = act->empresas;

else

ants->sig = acts->sig;

free (acts);

acts = ants->sig;

else

ants = acts;

acts = acts->sig;

if (act->empresas == NULL) // determinar lo

if (act == *LC)

*LC = ant;

ant->sig = act->sig;

else

ant->sig = act->sig;

free (act);

fclose (arch);

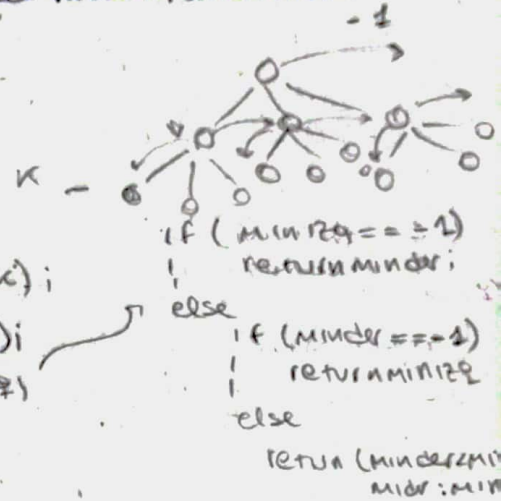


- ③ Se tiene un árbol N -ario de enteros positivos, hallar mediante una función recursiva int el valor de la menor clave no hoja del nivel k , si no existe devolver -1

```

int claemenor (arbol a, Posicion p, int nact, int k) {
    int min12, min2; Posicion ci;
    if (!Nulo(p)) {
        if (nact < k) {
            min12 = claemenor(a, HijoMasIzq(p, a), nact + 1, k);
            min2 = claemenor(a, HijoMasDer(p, a), nact + 1, k);
            (min12 > min2) ? return min2 : return min12;
        } else if (nact == k) {
            min = 1000;
            while (!Nulo(p)) {
                if (!Nulo(HijoMasIzq(p, a)) && info(p, a) < min)
                    min = info(p, a);
                p = HijoMasDer(p, a);
            }
            return min;
        } else
            return -1;
    }
}

```



- ④ Se tiene un grafo G con N vertices y aristas ponderadas almacenado en la matriz inferior de una matriz de adyacencia M y una Cola en la que cada elemento se identifica $id1$ e $id2$ ($1 \leq N$) y costo ($\text{costo} > 0$)

(a) Desarrollar en C solo los elementos que verifiquen que $id1$ e $id2$ estén unidos x veces con un costo a lo sumo costo

(b) asumiendo que C está implementado de nodo circular escribir el tipo y los operadores utilizados en la solución

```

void Eliminal (TCola c, int A[][MAX]) {
    TElemento x;
    TCola aux;
    while (!Vacia(c)) {
        sacar(c, &x);
        if (A[x.id1][x.id2] < x.costo)
            poner(aux, x);
    }
    while (!Vacia(aux)) {
        sacar(aux, &x);
        poner(c, x);
    }
}

```


totalizador 12/7/2021

① VOF

- ① la implementación de una cola circular permite aprovechar los espacios libres que van quedando al insertar y quitar elementos de esta. ^{la implementación sigue siendo estática solo se va cambiando los índices para aprovechar los espacios vacíos del vector}
- ② si luego de una rotación simple sobre un AVL que se había desbalanceado, el mismo sigue estando desbalanceado implica que debió haberse aplicado una rotación compuesta. ③ un AVL puede haberse desbalanceado al haber insertado o eliminado una clave, en el caso de la eliminación puede provocar que se realice más de una rotación

② Listas simples de entradas a un partido

{ Cod Partido (6) → ordenada
Capacidad Estadio
Entradas vendidas
Sublista entradas

{ Apellido y nombre del jugador de la compra
Cont. entradas
Mail del contacto
tipo entrada (A/B/C) → ordenada x este criterio

```
typedef struct nodo {  
    char CodPartido[7];  
    long int. Capacidad, contEntradas;  
    Sublista Entradas;  
    struct nodo *sig; } nodo
```

```
typedef struct nodo1 {  
    char NA[6], TIPO;  
    int Cont;  
    char Mail[24];  
    struct nodo1 *sig; }  
nodo1;
```

③ En el archivo COMPRAS.DAT que posee los datos necesarios para la compra, procesar el archivo sabiendo que el año es del 401, suponiendo se debe de almacenar una cola las ventas no realizadas guardando Apellido y nombre, Mail y partido

```
typedef struct {  
    char CodPartido[7]; NA[6], Mail[24], TIPO;  
    int Cont; } Reg
```

```
typedef struct {  
    char NA[6], Mail[24];  
    CodPartido[7]; } Telemtoc;
```

void procesarArch (Tlista L, Tcola *C)

FILE * Arch; Reg compra;
Telemtoc * n; ^{nuevo to}
Sublista act, ant; Tlista aux;

if (L != NULL && (Arch = fopen("COMPRAS.DAT", "rb")) != NULL)

while (fread(&compra, sizeof(Reg), 1, Arch) == 1)

aux = L;
while (aux != NULL && strcmp(aux->codPartido, compra.CodPartido) < 0)

aux = aux->sig;

if (aux != NULL && strcmp(aux->codPartido, compra.CodPartido) == 0) // se encontró partido

if (aux->ContEntradas + compra.Cont < aux->Capacidad * 0.4) // hay lugar

aux->ContEntradas += compra.Cont;

act = aux->Entradas; ant = NULL;

while (act != NULL && act->TIPO < compra.TIPO)

ant = act;

act = act->sig;

nuevo = (Sublista) malloc(sizeof(nodo1));

strcpy(nuevo->NA, compra.NA);

strcpy(nuevo->Mail, compra.Mail); nuevo->Cont = compra.Cont;


```

nuevoito → tipo ← compra.tipo;
IF (act == aux → Entradas) {
    nuevoito → sig = act;
    aux → Entradas = nuevoito;
}
else {
    act → sig = nuevoito;
    nuevoito → sig = act;
}
else {
    strcpy(x.mail, compra.mail);
    strcpy(x.codpartido, compra.codpartido);
    strcpy(x.NA, compra.NA);
    ponerC(x);
}
}
}
else {
    printf("Error al abrir el archivo o lista inexistente");
}
}

```

③ Se tiene un árbol binario AB proveniente de la transformación de un árbol N-ario AN, arboles de enteros. Hallar e informar si se verifica que la suma de las claves de cada uno de los árboles del bosque que se encuentra en alguna hoja del AN

```

int verifica (arbol B AB, arbol N AN) {
    int suma, cumple;
    cumple = 1;
    while (AB != NULL && cumple) {
        suma = AB → dato + suma(AB → der);
        cumple = Esta(AN, raíz(AN), suma);
        AB = AB → der;
    }
    return AB == NULL;
}

```

```

int suma(arbol B AB) {
    IF (!AB)
        return AB → dato + suma(AB → der)
        + suma(AB → der);
    else
        return 0;
}

```

```

int Esta(arbol N AN posición p, int x) {
    posición c;
    int cumple;
    IF (!Nulo(p)) {
        cumple = 0;
        c = hijoMaximo(p, AN);
        IF (Nulo(c) && info(p) == x)
            cumple = 1;
        else
            while (!Nulo(c) && !cumple) {
                IF (Nulo(hijoMaximo(c, AN)) &&
                    info(c, AN) == x)
                    cumple = 1;
                else {
                    c = hermanoDer(c, AN);
                }
            }
        return cumple;
    }
    else
        return 0;
}

```

④ void AMM (Tusta A[], int *maxi, int *maxj, int Arbol[], int *max, int *costo, int N) {
 Tusta aux; int k;
 for (k=0; k < N; k++) {
 aux = A[k];
 while (aux != NULL) {
 IF (aux → AMM) *costo += aux → costo;
 Arbol[k][aux → vertice] = aux → costo;
 IF (aux → costo > *max) {
 *max = aux → costo;
 *maxi = k; *maxj = aux → vertice;
 }
 aux = aux → sig;
 }
 }
}

totalidad 13/7/2020

① Indicar V o F

Ⓐ Para eliminar un nodo de una lista circular ordenada es necesario considerar como caso particular que sea el primero

Ⓑ Los valores obtenidos como resultado de la aplicación del algoritmo de Floyd puede también ser conseguidos x el algoritmo de Prim Ⓔ

Ⓐ ^{NO} Si se modifica el valor del campo siguiente de la cabeza de la lista pero no se modifica la cabeza de la lista

Ⓑ El algoritmo de Floyd da como resultado una matriz con las caminos más cortos entre todos los pares de vértices mientras que Prim encuentra el Árbol de menor costo total. Por lo tanto Ⓔ

② Dadas una cola de enteros (desordenados) y un Árbol B, eliminar de la cola a aquellos elementos que se encuentren en $[x, y]$ y estén en el Árbol B

Ⓐ Suponer que la cola es circular, escribir el tipo de cola y desarrollar los operadores requeridos en a)

void elimina (Tcola *c, int x, int y, arbol a)

Tcola aux;

Telemento aux;

while (!vacia(*c))

saca(c, &aux);

if (!(aux > x && aux < y) && !esta(a, aux))

poner(c, aux);

}

while (!vacia(aux))

saca(aux, &aux);

poner(c, aux);

}

}

int esta(arbol a, int x)

if (a == NULL)

if (x == a->dato)

return 1;

else

if (a->der > x)

return esta(a->der, x);

else

return esta(a->izq, x);

else

return 0;

}

Ⓐ Typedef int Telemento

typedef struct

TElemento { dato [MAX];

int pri, ult; } Tcola

void poner(Tcola *c, Telemento x)

int vacia(Tcola *c)

return c->pri == -1;

void saca(Tcola *c, Telemento *x)

if ((*c).pri == -1)

*x = (*c).datos[(*c).pri];

if ((*c).pri == (*c).ult)

inicial(c);

else

if ((*c).pri == MAX-1)

(*c).pri = 0;

else

(*c).pri++;

}


```

void Poner (Toda *c; TElemento x) {
    if (!((*c).ver == -MAX - 1 || ((*c).ver + 1 == (*c).pri))) {
        if ((*c).pri == -1) {
            (*c).pri = 0;
            (*c).ver = 0;
        }
        else {
            if ((*c).ver == MAX - 1)
                (*c).ver = 0;
            else
                (*c).ver += 1;
        }
        (*c).datos[(*c).ver] = x;
    }
}

```

③ LDE de alumnos con materias pendientes x aprobar

```

typedef struct nodo {
    char CDN[5]; // ordenada
    int intentos, APOC;
    float Prom;
    struct nodo * sig; // nodo
}
typedef struct nodo * sublista;

```

```

typedef struct nodo {
    char lesa[5], facu[5];
    sublista materias;
    nodo * sig;
}
typedef struct {
    nodo * pri, * ver; // + Listad
}

```

pendientes por aprobar = facultades

```

void Actualiza (TListad *LD, TListac *LC, int nota, char les[5], ...) {
    sublista acts, ants;
    TListad actD;
    TListac nuevante, actc;

    actD = (*LD).pri;
    while (actD != NULL && (strcmp(actD->les, les) != 0 && strcmp(actD->facu, facu) != 0)) {
        actD = actD->sig;
    }

    if (actD != NULL && (strcmp(actD->les, les) == 0 && strcmp(actD->facu, facu) == 0)) {
        acts = actD->materias;
        ants = NULL;
        while (acts != NULL && strcmp(acts->CDN, CDN) != 0) {
            ants = acts;
            acts = acts->sig;
        }
        if (acts != NULL && strcmp(acts->CDN, CDN) == 0) { // se controla materia
            if (nota >= 4 || (nota < 4 && acts->intentos == 2)) { // tengo que eliminar
                if (acts == actD->materias) { // si es la primera
                    actD->materias = acts->sig;
                }
                else {
                    ants->sig = acts->sig;
                }
            }
            if (strcmp(actD->facu, "ING") == 0 && nota < 4) {
                nuevo = (TListac) malloc (sizeof(nodo));
                strcpy (nuevo->les, les);
                strcpy (nuevo->CDN, CDN);
                strcpy (nuevo->facu, facu);
            }
        }
    }
}

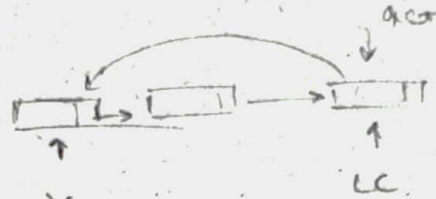
```



```

IF (*LC == NULL) {
    *LC = nuevo;
    (*LC) -> sig = nuevo;
}

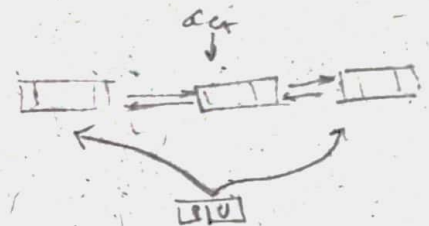
```



```

else if (strcmp((*LC) -> leg, leg) < 0) {
    nuevo -> sig = (*LC) -> sig;
    (*LC) -> sig = nuevo;
} else {
    antc = *LC; accc = (*LC) -> sig;
    while (strcmp(leg, accc -> leg) > 0) {
        antc = accc;
        accc = accc -> sig;
    }
    ant -> sig = nuevo;
    nuevo -> sig = accc;
}

```



```

IF (actD -> materias == NULL) {
    IF (actD == (*LD).pri) {
        (*LD).pri -> sig -> ant = NULL;
        (*LD).pri = (*LD).pri -> sig;
    } else IF (actD == (*LD).vet) {
        (*LD).vet -> ant -> sig = NULL;
        (*LD).vet = (*LD).vet -> ant;
    } else {
        actD -> ant -> sig = actD -> sig;
        actD -> sig -> ant = actD -> ant;
    }
}

```

```

free(actD);

```

```

free(acts);

```

```

else

```

```

IF (nota < 4)

```

```

    acts -> intentos++;

```


totalizado 10/2020

Indicador

- ① una cola circular es una cola estática o dinámica que sirve para aprovechar los espacios libres que quedan al efectuar eliminaciones (F) son implementaciones estáticas
- ② El algoritmo de Floyd tiene orden logarítmico por lo que es preferible que se ejecute en lugar de N veces Dijkstra (F) el orden del algoritmo de Floyd es justamente N veces Dijkstra es lo mismo que hacer N veces Dijkstra

② Nota simple en la vida

Num. Barrio (1 a 10) o duende Sublista de Construcciones DNI Propietario (cada 1 puede repetirse) Nro de lote (1 a 300) M² construido ↓ duende o barrio? Pila Nro de Barrio Nro de lote M² maximo construido permitidos

① Generar una LDE con los lotes relevados que hayan excedido lo que pueden construir

DNI no se repite → ordenado

M² total Excedidos

Cont. de lotes en los que se ha excedido

Los lotes que no se han excedido que no se han excedido se van a ir en la Pila

- ① El propietario con DNI P abandonará la multa correspondiente, eliminando de la lista doble indicando cuanto debe abonar, considerando que la multa es de \$K. Por cada metro excedido, más un 10% si tiene más de 5 construcciones excedidas

```
while (!vacio(+P)) {
    Saca P(P, R);
    act = L;
    while (act != NULL && strcmp(act->nBarrio, R.Barrio) != 0) // Busco el Barrio
        act = act->sig;
    if (act != NULL && strcmp(act->nBarrio, R.Barrio) == 0) // lo encuentre
        act = act->construcciones;
    while (act != NULL && act->nLote != R.NLote) // Busco el nro de lote de la
        act = act->sig; // Pila en la sublista del Barrio
    if (act != NULL && act->nLote == R.NLote) // lo encuentre
        if (act->m2const > R.M2max) { // si excedi
            nuevo = (nodo*) malloc(sizeof(nodo));
            strcpy(nuevo->DNI, act->DNI);
            nuevo->m2tot = act->m2const - R.M2max;
            nuevo->contexc = 1; nuevo->sig = NULL; nuevo->act = NULL;
            if (strcmp(nuevo->DNI, (*LD).Pri->DNI) < 0) {
                nuevo->sig = (*LD).Pri;
                (*LD).Pri->act = nuevo;
                (*LD).Pri = nuevo;
            }
        }
}
```

```
else{
```

```
if(strcmp((L*LD).pri->DNI, act->DNI) < 0){
```

```
    (*LD).ver->sis = nuevo;
```

```
    nuevo->ant = (*LD).ver;
```

```
    (*LD).ver = nuevo;
```

```
else{
```

```
    actD = (*LD).pri;
```

```
    while(actD != NULL && strcmp(actD->DNI, act->DNI) < 0){
```

```
        actD = actD->sis;
```

```
    if(actD != NULL && strcmp(actD->DNI, act->DNI) == 0){
```

```
        actD->N2TOTEXC = nuevo->N2TOTEXC;
```

```
        (actD->cantex)++;
```

```
        Free(nuevo);
```

```
    }else{
```

```
        act->ant->sis = nuevo;
```

```
        nuevo->ant = act->ant;
```

```
        act->ant = nuevo;
```

```
        nuevo->sis = actD;
```

```
    }
```

```
    }
```

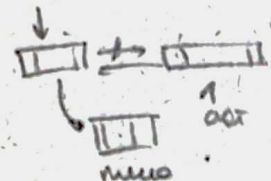
```
    Free(actD);
```

```
    } //completa el arbol (1 vuelta)
```

```
while(!strcmp(aux))
```

```
    scanf("%d", &x);
```

```
    printf("%d", x);
```



ii) void elimina (tlistaD *LD, char P[10], float K)

```
if(strcmp((LD).ver->DNI, P) == 0){
```

```
    act = (*LD).ver;
```

```
    (*LD).ver = act->ant;
```

```
    meta = act->N2TOT * K;
```

```
    if(act->cantex > 5)
```

```
        meta = 1.1;
```

```
    printf("Severa alarma: %.1f", meta);
```

```
else{
```

```
    act = (*LD).pri;
```

```
    while(act != NULL && strcmp(act->DNI, P) < 0){
```

```
        act = act->sis;
```

```
    if(act != NULL && strcmp(act->DNI, P) == 0){
```

```
        if(act == (*LD).pri)
```

```
            (*LD).pri = act->sis;
```



```
else{
```

```
    act->ant->sis =
```

```
    act->sis;
```

```
    act->sis->ant =
```

```
    act->ant;
```

```
    }
```

```
    Free(act);
```


- ③ se tiene un árbol N -ario de árboles enteros, determinar si todas las afuniones de k en AN se dan en niveles pares

```
int nivelesPares(arbol a, posición p, int k, int nact)
```

```
    parámetro
```

```
    if (!Nulo(p))
```

```
    { if (info(p.a) == k)
```

```
        if (nact % 2 == 0)
```

```
            return nivelesPares(a, hijoNro1(p.a), k, nact+1) ||
```

```
            nivelesPares(a, hermanoDer(p.a), k, nact);
```

```
        else
```

```
            return 0;
```

```
    } else
```

```
        return nivelesPares(a, hijoNro1(p.a), k, nact+1) || nivelesPares(a, hermanoDer(p.a), k, nact);
```

```
    } else
```

```
        return 1;
```

- ④ Dado un grafo con aristas ponderadas implementado en una lista de adyacencia, onde que cada nodo de las listas adyacentes del vertice se tiene el peso y 150 según la arista pertenece al AMM, desarrollar una función que permita determinar el costo del AMM y la cantidad de aristas que lo conforman

```
void CostoAMM(TLISTA A[], int *contaristas, int *costoTot, int N)
```

```
    int i;
```

```
    TLISTA act;
```

```
    for (i=0; i<N; i++)
```

```
        act = A[i];
```

```
        while (act != NULL)
```

```
            if (act->AMM)
```

```
                (*contaristas)++;
```

```
                (*costoTot) += act->peso;
```

```
            act = act->sig;
```

```
}
```

totalidad 24/8/2024

① V of

- ② Para eliminar un nodo de una lista circular ordenada no es necesario considerar como caso particular que sea el primero ① no es el primero la cabeza es el primer nodo de la lista
- ③ Los valores obtenidos como resultado de la aplicación del algoritmo de Floyd pueden ~~ser~~ ^{amb} conseguidos con el de Dijkstra o Prim ④ solo con Dijkstra

② se tiene un grafo conexo con aristas ponderadas $G=(V,E)$ con $|V|=N$ almacenando en una matriz de adyacencia y una cola C con valores enteros no repetidos. Desarrollar una solución para sacar de C solo los valores que no representen el grado de entrada en ningún vértice de G

void Elimina(T cola *C, int A[][MAX], int N)

int S[] = {0}, i, j

T elemento x; T cola aux;

for (i=0; i<N; i++)

for (j=0; j<N; j++)

if (A[i][j] != 0)

S[j]++;

while (!vacia(*C))

saca(C, &x);

i=0;

while (i<N && S[i] != x)

i++;

if (S[i] == x)

pus(C, &aux, x);

};

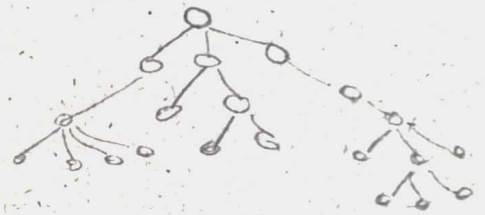
while (!vacia(aux))

saca(aux, &x);

pus(C, x);

}

	1	2	3
1	0	7	
2	8	3	



③ se tiene un árbol N -ario de enteros, definir las hojas involucradas y

④ Generar una lista doblemente enlazada con las claves que estén en niveles impares y sea hojas, la lista debe estar ordenada en orden ascendente por nivel

⑤ Eliminar de la lista, las claves que se encuentren en el nivel K (dato de entrada) informar la cantidad de claves eliminadas

void generaLista(T lista *L, arbol a, Posicion p, int nact)

Posicion *nuevo, aux;

if (!Nulo(p))

if (HijaMasIzq(p, a) == NULL && nact % 2 == 1)

{ nuevo = (Posicion *) malloc (sizeof(Posicion));

nuevo->nivel = nact;

nuevo->clave = info(p, a); nuevo->sig = NULL; nuevo->ant = NULL;


```
if ((*LD).Pri == NULL)
```

```
    (*LD).Pri = nuevo;
```

```
    (*LD).ult = nuevo;
```

```
    }
```

```
else if (nuevo->nivel < (*LD).Pri->nivel)
```

```
    (*LD).Pri->ant = nuevo;
```

```
    nuevo->sig = (*LD).Pri;
```

```
    (*LD).Pri = nuevo;
```

```
    }
```

```
else
```

```
    if (nuevo->nivel == (*LD).ult->nivel)
```

```
        (*LD).ult->sig = nuevo;
```

```
        nuevo->ant = (*LD).ult;
```

```
        (*LD).ult = nuevo;
```

```
    else if
```

```
        act = (*LD).Pri;
```

```
        while (act != NULL && act->nivel < nact)
```

```
            act = act->sig;
```

```
        act->ant->sig = nuevo;
```

```
        nuevo->ant = act->ant;
```

```
        nuevo->sig = act;
```

```
        act->ant = nuevo;
```

```
    }
```

```
}
```

```
GeneraLista(LD, a, &ismasizq(p,a), nact+1);
```

```
GeneraLista(LD, a, &remasder(p,a), nact);
```

```
}
```

```
⑥ void Elimina (TLista *LD, int k)
```

```
    int cantelim;
```

```
    Node *act, elim;
```

```
    act = (*LD).Pri;
```

```
    while (act != NULL && act->nivel <= k)
```

```
        if (act->nivel == k)
```

```
            cantelim++;
```

```
            if (act == (*LD).Pri)
```

```
                (*LD).Pri = act->sig;
```

```
            (*LD).Pri->ant = NULL;
```

```
            free(act);
```

```
            act = (*LD).Pri;
```

```
        else
```

```
            if (act == (*LD).ult)
```

```
                (*LD).ult = act->ant;
```

```
                (*LD).ult->sig = NULL;
```

```
        free(act);
```

```
        act = (*LD).ult;
```

```
    else
```

```
        elim = act;
```

```
        act->ant->sig = act->sig;
```

```
        act->sig->ant = act->ant;
```

```
        act = act->sig;
```

```
        free(act);
```

```
    }
```

```
    else if nact != k
```

```
        act = act->sig;
```

```
    printf("La cantidad de eliminaciones son '%d' en '%d' cantelim", cantelim);
```